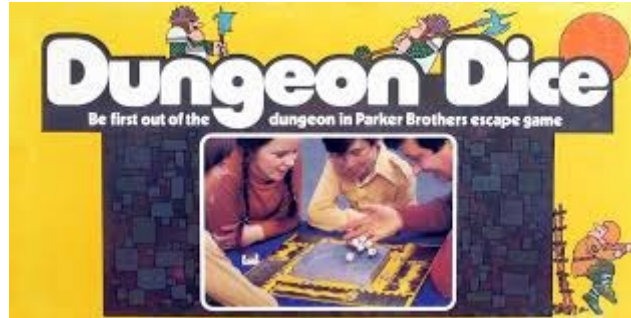


Department of Electrical and Computer Engineering

ECE 1400 Semester Project

The project for this semester is to program (first in Python then in C) a single-player version of the 1977 Parker Brothers® board game Dungeon Dice.



The basic idea is that you have to earn 8 tunnel cards to escape from the dungeon. (The board is artistically set up so the tunnel cards show your progress, but all that matters is that you earn 8 or more cards to win.)



You earn cards by taking turns with the dice. These dice are not your typical dice. Five of the faces are tools that would help you escape from a dungeon. They are a dagger, a shovel, a ladder, a lantern and a key. The sixth face is a guard, which, as you will see, is not always a good thing.



Players take turns with the dice and the first to acquire 8 cards is the winner. (Technically, the actual game is a little more complicated than that. There are some rules for challenging opponents that we will ignore since we are implementing a single-player game.)

A turn works like this:

1. Roll all six dice into the dungeon pit.

Repeat steps 2-6 until your turn ends.

2. Move any dice that match to the dungeon wall (or if you have one of each symbol you move all the dice to the wall).
3. If at any time there are 3 guards on the wall, your turn ends and you lose one tunnel card.
4. The dice on the wall are scored to see how many tunnel cards they are worth:

3-of-a-kind:	1 card (2 cards for 2 sets of 3)
4-of-a-kind:	2 cards
5-of-a-kind:	3 cards
6-of-a-kind:	4 cards
three pair:	1 card
one of each:	3 cards
5. Normally, if the total number of tunnel cards earned is non-zero at this point, you may collect those cards and end your turn (but there is an exception to this rule described in Step 7). Otherwise, re-roll the dice in the pit (or, if the pit is empty, total the number tunnel cards earned so far and include them in step 4 going forward, then re-roll all six dice).
6. After re-rolling the dice, if you have no matching dice to move to the wall, your turn ends and you get no tunnel cards.
7. Exception: If you have 2 guards on the wall and the pit is not empty, you are compelled to re-roll (i.e. you cannot stop and take your card(s)).

In the actual board game, turns rotate between players and the first one out wins, but in this single player game, only one player takes turns and the objective is to see how few turns you need to escape.

The requirements for this computer game are listed below:

1. The computer program shall maintain the number of turns, the number of cards the player has collected, the number of cards the player has earned so far in the current turn, a list of dice in the pit and a list of dice on the wall. Initially the program shall reset the number of turns and the number of cards the player has collected.
2. The program shall execute a series of turns (as described in requirements 3-9) and exit only when the player has collected a total of 8 or more cards.
3. As each turn commences, the number of earned cards (for this turn) shall be set to 0 and all 6 dice shall be rolled and be placed in the pit. The program shall output the turn number (starting with turn 1) and the number of cards collected (so far) using the following format example:
Turn #1. Total cards collected: 0.
4. After a turn has commenced, then again after each time the dice are re-rolled, the matching dice (if any) shall be moved from the pit to the wall. The status of the game shall then be displayed in the following format:
Wall: key(2), Lantern(2), Pit: Guard, Shovel, Cards: 0

(The pit portion may be omitted if the pit is empty. The numbers in parentheses tell how many of the named dice are on the wall. Note that the number of cards shown shall be -1 if there are 3 or more guards on the wall. Otherwise it shall be equal to the evaluation of the wall dice plus the number of cards earned earlier in the turn.)

5. If, after the status is displayed (Requirement 4), there are 3 guards on the wall, the program shall output:
 Too many guards – give back one card.
 and the turn shall conclude. The number of collected cards shall then be decremented if it is not already 0.
6. If, no dice are moved to the wall as described in Requirement 4, then, after the status is displayed, the program shall output:
 Failed to move any dice to the wall.
 and the turn shall conclude without the player collecting any cards.
7. If, after the status is displayed (Requirement 4), the player is compelled to roll again (see rules, above), the program shall prompt:
 You are required to roll again, press ENTER
 The program shall then wait for the user to press ENTER before re-rolling.
8. If, after the status is displayed (Requirement 4), the player has the option to end his turn or roll again, the program shall prompt:
 Would you like to roll again?
 Any response that starts neither with 'Y' nor 'y' shall be interpreted by the program as "No", which shall cause the turn to end and the earned cards from this turn to be added to the players collected cards.
9. If the conditions in Requirement 7 are met or the player's response in Requirement 8 is affirmative, the dice shall be re-rolled. If there are dice in the pit, only those dice shall be re-rolled. If there are no dice in the pit, the earned cards shall be augmented to include the dice on the wall and all of the dice shall be re-rolled. In either case, the procedure described in Requirement 4 shall be retriggered.
10. If, after a turn, the player has 8 or more cards, the program shall print a termination message using the format of the example, below:
 You have escaped after 5 turns!
11. For the python version, rolling a die must be implemented using `random.randint(0,5)` where 0 represents the "guard".
12. To make grading tractable, the python program you submit must seed the random number generator with 168. This should generate an initial roll with one guard and one each of the other 5 symbols.

Here are some random number seeds for debugging (Python only)

Seed	Result	Seed	Result	Seed	Result
0	3-of-a-kind	3	3-of-a-kind + a pair	117	5-of-a-kind
1	2 guards	10	3 pairs	151	4-of-a-kind
2	3 guards	11	2 three-of-a-kind	168	6 all different

For your python version, please make it clear where you seed your random number. I will need to modify the seed during my testing of your code.

Your program should consist of a single file (with a .py or .c extension depending on whether it is in Python or C). Submit your program file to CANVAS on or before the due date. The plagiarism checker will be enabled and high plagiarism or AI generation scores will result in my scrutiny and may result in a score as low as 0 points on the assignment.

Example Game (seed = 168):

Turn #1. Total cards collected: 0.

Wall: Guard(1), Lantern(1), Shovel(1), Dagger(1), Ladder(1), Key(1), Cards: 3

Would you like to roll again? y

Wall: Lantern(3), Ladder(3), Cards: 5

Would you like to roll again? y

Wall: Dagger(2), Ladder(2), Pit: Lantern, Key, Cards: 5

Would you like to roll again? y

Wall: Guard(2), Dagger(2), Ladder(2), Cards: 6

Would you like to roll again? n

Turn #2. Total cards collected: 6.

Wall: Shovel(3), Ladder(2), Pit: Dagger, Cards: 1

Would you like to roll again? n

Turn #3. Total cards collected: 7.

Wall: Dagger(2), Pit: Guard, Lantern, Shovel, Ladder, Cards: 0

You are required to roll again, press ENTER.

Wall: Dagger(2), Pit: Guard, Lantern, Shovel, Key, Cards: 0

Failed to move any dice to the wall.

Turn #4. Total cards collected: 7.

Wall: Lantern(2), Dagger(2), Pit: Guard, Ladder, Cards: 0

You are required to roll again, press ENTER.

Wall: Lantern(2), Dagger(3), Pit: Shovel, Cards: 1

Would you like to roll again? n

You have escaped after 4 turns!